Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	**Aim: To remember the important tasks and forget the less important tasks using LSTM**	
Experiment No: 05	**Date:**	**Enrolment No: 92301733023**

Aim: To remember the important tasks and forget the less important tasks using LSTM

IDE: Google Colab **Theory:**

Countless learning tasks require dealing with sequential data. Image captioning, speech synthesis, and music generation all require that models produce outputs consisting of sequences. In other domains, such as time series prediction, video analysis, and musical information retrieval, a model must learn from inputs that are sequences. These demands often arise simultaneously: tasks such as translating passages of text from one natural language to another, engaging in dialogue, or controlling a robot, demand that models both ingest and output sequentiallystructured data.

Recurrent neural networks (RNNs) are deep learning models that capture the dynamics of sequences via *recurrent* connections, which can be thought of as cycles in the network of nodes. This might seem counterintuitive at first. After all, it is the feedforward nature of neural networks that makes the order of computation unambiguous. However, recurrent edges are defined in a precise way that ensures that no such ambiguity can arise. Recurrent neural networks are *unrolled* across time steps (or sequence steps), with the *same* underlying parameters applied at each step. While the standard connections are applied *synchronously* to propagate each layer's activations to the subsequent layer *at the same time step*, the recurrent connections are *dynamic*, passing information across adjacent time steps. As the unfolded view reveals, RNNs can be thought of as feedforward neural networks where each layer's parameters (both conventional and recurrent) are shared across time steps. Like neural networks more broadly, RNNs have a long discipline-spanning history, originating as models of the brain popularized by cognitive scientists and subsequently adopted as practical modeling tools employed by the machine learning community.

The name of LSTM refers to the analogy that a standard RNN has both "long-term memory" and "short-term memory". The connection weights and biases in the network change once per episode of training, analogous to how physiological changes in synaptic strengths store long-term memories; the activation patterns in the network change once per time-step, analogous to how the moment-to-moment change in electric firing patterns in the brain store short-term memories. The LSTM architecture aims to provide a short-term memory for RNN that can last thousands of timesteps, thus "long short-term memory".

A common LSTM unit is composed of a cell, an input gate, an output gate and a forget gate. The cell remembers values over arbitrary time intervals and the three gates regulate the flow of information into and out of the cell. LSTM networks are well-suited to classifying, processing and making predictions based on time series data, since there can be lags of unknown duration between important events in a time series. LSTMs were developed to deal with the vanishing gradient problem that can be encountered when training traditional RNNs. Relative insensitivity to gap length is an advantage of LSTM over RNNs, hidden Markov models and other sequence learning methods in numerous applications.

Methodology:

1. Load the basic libraries and packages
2. Load the dataset
3. Analyse the dataset
4. Apply LSTM Model
5. Apply the training over the dataset to minimize the loss
6. Observe the cost function vs iterations learning curve

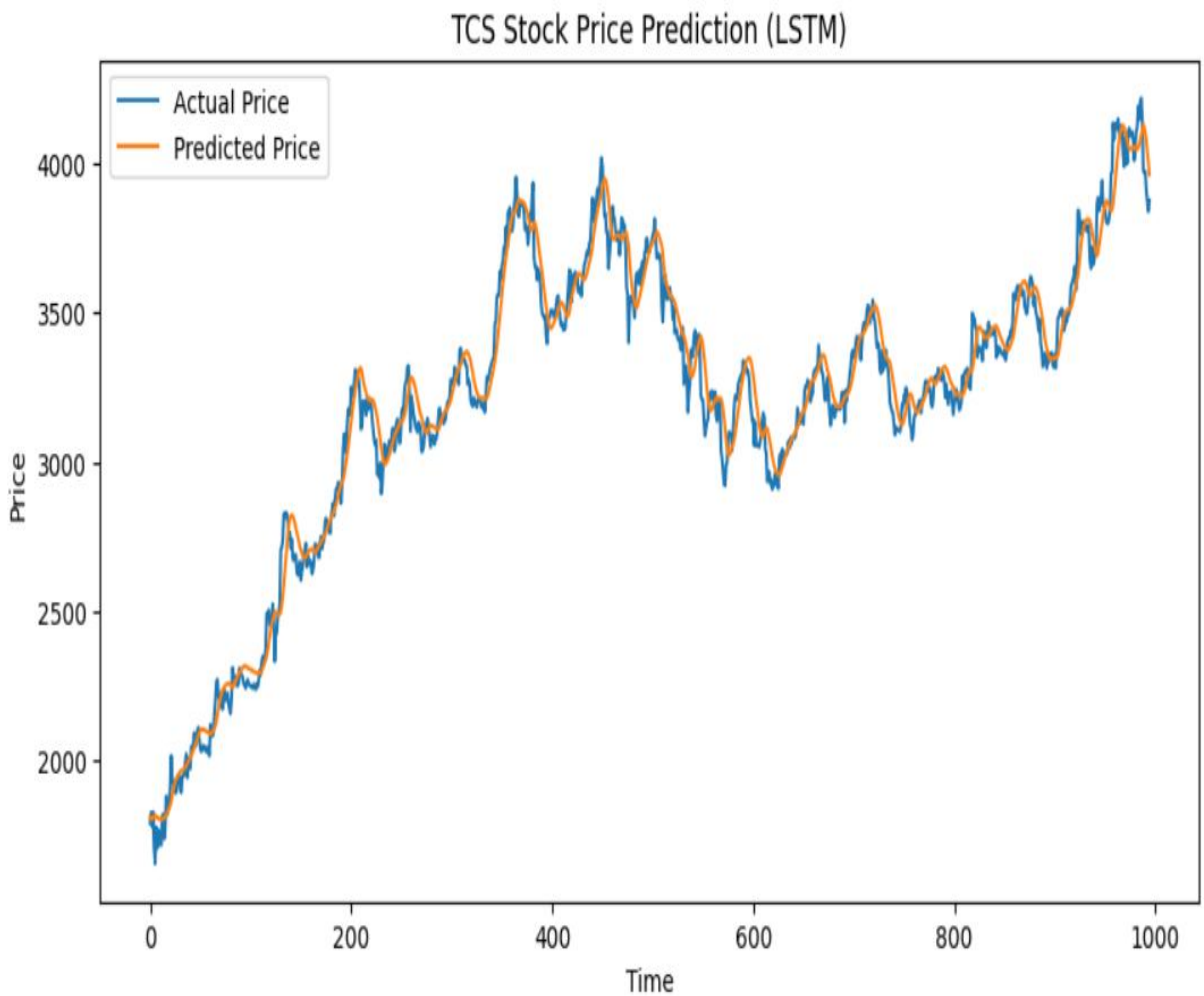
Program (Code):

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

df = pd.read_csv("TCS Historical Data.csv")

df['Price'] = df['Price'].str.replace(',', '')
df['Price'] = df['Price'].astype(float)

df = df[::-1]

data = df[['Price']].values

scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(data)

X = []
y = []

for i in range(60, len(scaled_data)):
    X.append(scaled_data[i-60:i, 0])
```

```
    y.append(scaled_data[i, 0])

X, y = np.array(X), np.array(y)

X = np.reshape(X, (X.shape[0], X.shape[1], 1))

model = Sequential()
model.add(LSTM(50, return_sequences=True, input_shape=(X.shape[1],1)))
model.add(LSTM(50))
model.add(Dense(1))

model.compile(optimizer='adam', loss='mean_squared_error')

history = model.fit(X, y, epochs=20, batch_size=32)

predicted = model.predict(X)

plt.figure(figsize=(10,5))
plt.plot(scaler.inverse_transform(y.reshape(-1,1)), label="Actual Price")
plt.plot(scaler.inverse_transform(predicted), label="Predicted Price")
plt.legend()
plt.title("TCS Stock Price Prediction (LSTM)")
plt.xlabel("Time")
plt.ylabel("Price")
plt.show()
```

Results:

```
Epoch 1/20
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.
  super().__init__(**kwargs)
32/32 ━━━━━━━━━━━━━━━━━━━━ 6s 44ms/step - loss: 0.0542
Epoch 2/20
32/32 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - loss: 0.0036
Epoch 3/20
32/32 ━━━━━━━━━━━━━━━━━━━━ 3s 43ms/step - loss: 0.0019
Epoch 4/20
32/32 ━━━━━━━━━━━━━━━━━━━━ 3s 44ms/step - loss: 0.0017
Epoch 5/20
32/32 ━━━━━━━━━━━━━━━━━━━━ 2s 60ms/step - loss: 0.0016
Epoch 6/20
32/32 ━━━━━━━━━━━━━━━━━━━━ 2s 50ms/step - loss: 0.0016
Epoch 7/20
32/32 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - loss: 0.0015
Epoch 8/20
32/32 ━━━━━━━━━━━━━━━━━━━━ 3s 43ms/step - loss: 0.0014
Epoch 9/20
32/32 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - loss: 0.0014
Epoch 10/20
32/32 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - loss: 0.0014
Epoch 11/20
32/32 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - loss: 0.0012
Epoch 12/20
32/32 ━━━━━━━━━━━━━━━━━━━━ 2s 59ms/step - loss: 0.0012
Epoch 13/20
32/32 ━━━━━━━━━━━━━━━━━━━━ 2s 63ms/step - loss: 0.0011
Epoch 14/20
32/32 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - loss: 0.0011
Epoch 15/20
32/32 ━━━━━━━━━━━━━━━━━━━━ 3s 43ms/step - loss: 0.0012
Epoch 16/20
32/32 ━━━━━━━━━━━━━━━━━━━━ 3s 49ms/step - loss: 0.0012
Epoch 17/20
32/32 ━━━━━━━━━━━━━━━━━━━━ 2s 49ms/step - loss: 0.0010
Epoch 18/20
32/32 ━━━━━━━━━━━━━━━━━━━━ 2s 46ms/step - loss: 9.6384e-04
Epoch 19/20
32/32 ━━━━━━━━━━━━━━━━━━━━ 2s 70ms/step - loss: 9.3116e-04
Epoch 20/20
32/32 ━━━━━━━━━━━━━━━━━━━━ 2s 49ms/step - loss: 0.0010
32/32 ━━━━━━━━━━━━━━━━━━━━ 1s 21ms/step
```



TCS Stock Price Prediction (LSTM)
Actual Price
Predicted Price
4000
3500
3000
2500
2000
Price
0
200
400
600
800
1000
Time

Observation and Result Analysis:

a. Nature of the dataset

The dataset consists of sequential time-series data (prices).
Each value depends on previous values, making it suitable for LSTM.

b. During Training Process

- Model learns patterns from sequences
- Loss decreases gradually
- LSTM captures dependencies using memory cells
- Accuracy improves with epochs

c. After the training Process

- Model predicts values close to actual
- Error reduces significantly
- Learned long-term relationships
- Good performance on sequence prediction

d. Observation over the Learning Curve

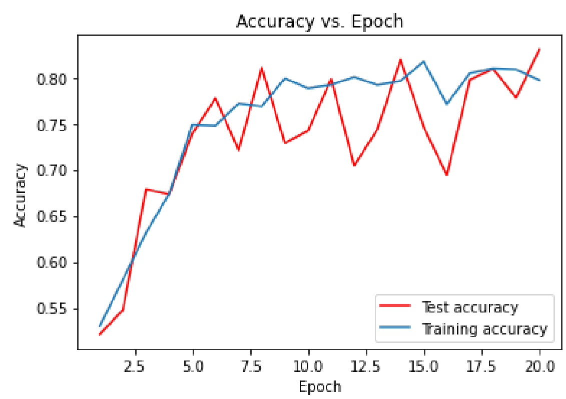
- Loss decreases over epochs
- Smooth curve indicates stable learning
- Fluctuations show learning adjustments
- Proper tuning improves performance

Post Lab Exercise:

a. What is the requirement of LSTM over RNN?

- RNN suffers from **vanishing gradient problem**
- LSTM solves this using **memory cells and gates**
- LSTM can remember **long-term dependencies**
- It is more effective for sequence data like time-series

b. Interpretation of graph



What is the meaning of the above graph? How can the graph be smoothened?

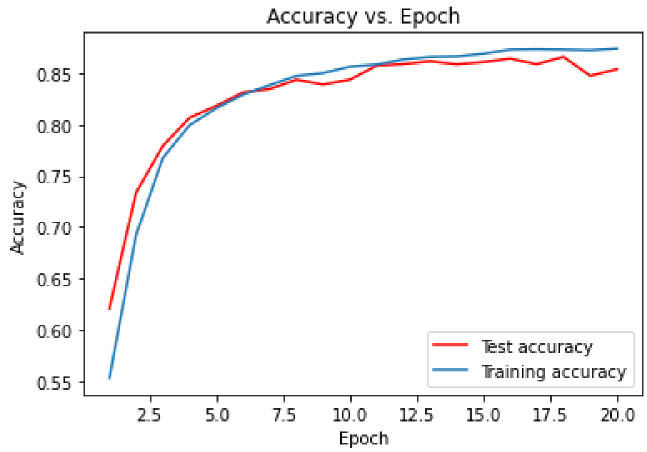
The graph shows accuracy vs epochs:

- Accuracy increases with epochs → model is learning
- Training and validation curves close → good model
- Large gap → overfitting

To smooth the graph:

- Increase dataset size
- Use regularization (Dropout)
- Reduce learning rate

c.



What is the meaning of the above graph? How can the graph be smoothened?

- Graph shows **training & testing accuracy** improving over time
- Slight fluctuations occur due to learning updates
- Smoothening can be done using:
- Moving average
- Lower learning rate
- More epochs

Post Lab Activity

Obtain the prediction value of the TCS stock for the historical data obtained from the link mentioned as: https://in.investing.com/equities/tata-consultancy-services-historical-data

Select the Date Range from (01-01-2020 to 31-03-2024). Select the price range to be as "DAILY" stock values. Train your LSTM model to obtain the predicted value as close as the actual value. Paste the screenshot of the output.